Cricket Team
Performance Analyzer

Project Documentation

Developed by

Pranay Jha

B.Tech — Computer Science and Engineering

Powered by Python & Streamlit

Table of Contents

1. Project Overview
2. Motivation & Problem Statement
3. Objectives
4. Dataset Structure
5. System Architecture
6. Data Pipeline
7. Analytical Modules
8. Tech Stack
9. Version History
10. Known Limitations
11. Future Scope

1. Project Overview

The Cricket Team Performance Analyzer is a data analytics tool designed to evaluate and visualize the performance of a cricket team across one or more matches. The tool accepts structured match-level player data as input and produces a comprehensive set of batting analytics, bowling analytics, and a team-level performance summary.

The project was first developed as a Python-based exploratory analysis in a Jupyter Notebook environment (Google Colab) using real IPL 2024 match data for Royal Challengers Bengaluru (RCB). It was later transformed into a fully interactive web application using Streamlit, with an improved user interface, enhanced data validation, caching, and multi-match support.

In its current version (2.1), the tool is organised into five tabs — Dataset, Batting Analytics, Bowling Analytics, Player Analytics, and Summary — and ships with a realistic 10-match example dataset (match_data_fixed.csv) that passes every validation rule. The tool currently supports the T-20 format only.

The tool is intended for local cricket teams, school and college tournament organizers, and small-scale cricket competitions where professional analytics infrastructure is unavailable, but data-driven insights would still be valuable.

2. Motivation & Problem Statement

In professional cricket leagues such as the IPL, BCCI, or international T20 tournaments, performance analysis is handled by dedicated analytics teams using advanced tools and large datasets. However, at the grassroots level — local matches, gully cricket, inter-college tournaments, and community leagues — player performance is almost always evaluated manually, subjectively, or based solely on memorable moments during a match.

This leads to several recurring problems:

- Player contributions are judged on perception rather than data, leading to biased assessments.
- Awards such as Best Batsman or Best Bowler are decided without considering complete statistics.
- Bowling performance is typically judged only on wickets taken, while economy rate — an equally important metric — is ignored.
- There is no structured system to track individual or team performance across multiple matches.
- Coaches and captains lack objective data to make informed decisions about batting order, bowling rotations, or team composition.

This project addresses these problems by providing a simple, structured, and data-driven performance evaluation system that any team can use without needing technical expertise beyond filling in a spreadsheet.

3. Objectives

The core objectives of this project are:

- Build a clear and unbiased cricket performance analysis system that replaces manual observation with quantitative metrics.
- Measure individual batting contribution through runs scored, balls faced, and strike rate.
- Evaluate bowling impact through wickets taken and economy rate, treating both metrics as equally important.
- Measure player consistency across multiple matches using a statistical consistency score.
- Compare performances between different player roles — batsmen, bowlers, and all-rounders — and understand their relative contributions to the team.
- Identify phase-wise weaknesses in batting by analysing when the team loses the most wickets (powerplay, middle overs, or death overs).
- Provide a fair, automated statistical summary for team evaluation after each match or series of matches.
- Make the tool accessible to non-technical users through a clean, file-upload-based web interface.

4. Dataset Structure

The tool accepts data in .csv or .xlsx format. Each row in the dataset represents one player's performance in one match. A dataset can contain data from a single match or multiple matches combined.

4.1 Column Definitions

Column Name	Description
Match_No	Integer. The match number this row belongs to. Must be filled for every row. Use 1 for the first match, 2 for the second, and so on.
Player_Name	Text. The full or short name of the player. Must be consistent across all matches — the same player must be spelled identically in every row.
Role	Text. Accepted values: batsman, bowler, all-rounder, wicketkeeper. Capitalisation does not matter. If blank, auto-filled with the most common role. Wicketkeeper is treated as batsman for batting analytics.
Batting_Position	Integer (1–11). The position at which the player came in to bat. Must be filled for every row. No two players in the same match can share the same batting position.
Batting_Start_Over	Numeric. The over number when the player began their innings. Leave blank if the player did not bat. Supports fractional values (e.g., 6.3 = third ball of over 6). Must be ≤ Out_Over.
Out_Over	Numeric or blank. The over number when the player was dismissed. Leave blank or write not-out if not dismissed. Cannot exceed 20.
Balls_Played	Integer. Total balls faced by the player. Must be filled if Batting_Start_Over is filled.
Runs_Scored	Integer. Total runs scored. If dismissed for zero, enter 0 — do not leave blank.
Overs_Bowled	Numeric. Total overs bowled. Digit after decimal = balls in incomplete over (e.g., 3.4 = 3 overs and 4 balls). Valid ball values are 1–5 only. Leave blank or 0 if did not bowl.
Runs_Given	Integer. Total runs conceded while bowling. Must be filled if Overs_Bowled is filled.
Wickets_Taken	Integer. Number of wickets taken while bowling. Run-outs should not be credited here. The total wickets taken by all players in a single match cannot exceed 10.

4.2 Dataset Constraints

- A single match cannot have more than 11 players.
- The dataset should not contain more than 22 unique players at a time (equivalent to 2 complete teams).
- Out_Over cannot exceed 20 — only T20 format is supported.
- The total wickets taken by all players in a single match cannot exceed 10.
- Batting_Start_Over must not be greater than Out_Over for any player.

5. System Architecture

The project is structured across two layers, evolving from an exploratory notebook to a production-grade web application.

5.1 Layer 1 — Notebook (Exploratory Phase)

The original analysis was conducted in a Google Colab Jupyter Notebook. This layer handles data loading, validation, cleaning, feature engineering, and static chart generation. The notebook uses three example IPL 2024 match datasets (RCB) concatenated into a single DataFrame. All outputs are rendered inline within the notebook.

5.2 Layer 2 — Streamlit Web Application (Production Phase)

The notebook logic was re-engineered into a modular Streamlit application. This layer adds a file upload interface, session state management, interactive player-level filtering, multi-match and multi-file support, and a fully themed dark dashboard with five tabbed sections.

Application Flow

- User uploads file(s) via the web interface.
- Data concatenation — single or multiple files are merged into one DataFrame.
- Data validation — 10+ rule checks run; errors are raised on failure.
- Data cleaning and feature engineering — results are cached with @st.cache_data.
- The processed DataFrame is stored in st.session_state and persists across tab switches.
- Batting Analytics, Bowling Analytics, Player Analytics, and Summary tabs render from cached results.

Key Architectural Decisions

- **st.session_state** — Persists the processed DataFrame across tab switches, preventing redundant re-processing on every interaction.
- **@st.cache_data** — Ensures the same uploaded file is not reprocessed on every Streamlit rerun.
- **Global constants** — All chart size constants and colour palettes are defined globally for easy maintenance of visual consistency.

- **Typed exceptions** — The validation function raises typed ValueError exceptions that are caught and displayed as user-facing error messages, keeping the UI clean.
- **st.fragment** — The upload section, the Player Analytics tab, and the per-player dropdowns are fragments, so changing a selection re-runs only that section instead of re-drawing every chart on every tab. On older Streamlit versions the app falls back to normal reruns.
- **Shared cached statistics** — compute_team_stats() builds all team-level tables once per dataset and shares them across the Batting, Bowling, and Summary tabs; compute_player_match_stats() does the same for Player Analytics.
- **Reusable chart helpers** — bar_chart(), line_chart(), and pie_chart() replace repeated plotting code and keep styling consistent across tabs.
- **Single refresh after upload** — validation messages are stored in session state and the app refreshes once after Confirm Upload so every tab sees the new dataset.

6. Data Pipeline

Step 1 — Data Loading

The user uploads one or more .csv or .xlsx files through the Streamlit file uploader. Multiple files are concatenated into a single DataFrame using pd.concat with ignore_index=True.

Step 2 — Data Validation

The data_validation() function runs 10+ rule checks before any cleaning takes place. These include:

- Out_Over cannot exceed 20 (T20 format constraint).
- No match can contain more than 11 players.
- No more than 22 unique players across the entire dataset.
- The total wickets taken by all players in a single match cannot exceed 10.
- Every row must have a Match_No and a Batting_Position.
- No row with player statistics can have a missing Player_Name.
- Batting statistics cannot be partially filled — if Batting_Start_Over is present, Balls_Played and Runs_Scored must also be present. Out_Over may be left blank or written as not-out for a batter who was not dismissed.
- Batting_Start_Over cannot be greater than Out_Over for any player.
- Bowling statistics cannot be partially filled — if Overs_Bowled is present and greater than 0, Runs_Given and Wickets_Taken must also be present.

If any check fails, a descriptive error is raised and displayed to the user. Processing does not proceed until all checks pass.

Step 3 — Data Cleaning

Once validation passes, the following cleaning operations are applied on a copy of the original DataFrame:

- Role values are stripped of whitespace and lowercased for consistency.
- Missing Role values are filled with the mode of the column.
- wicketkeeper role is remapped to batsman.
- Batting_Start_Over, Out_Over, Balls_Played, Runs_Scored, Overs_Bowled, Runs_Given, and Wickets_Taken nulls are filled with 0.
- Out_Over is coerced to numeric, converting not-out strings to NaN and then to 0.

Step 4 — Feature Engineering

Six new columns are derived after cleaning:

- **Batted** — Boolean. True if Batting_Start_Over > 0, indicating the player actually batted in that match.
- **Was_Out** — Boolean. True if Out_Over > 0, indicating the player was dismissed.
- **Strike_Rate** — Float. Calculated as (Runs_Scored / Balls_Played) × 100, rounded to 2 decimal places. Set to 0.0 for players who faced no balls.
- **Economy_Rate** — Float. Calculated as Runs_Given / (Balls_Bowled / 6), so fractional overs are converted exactly (e.g., 3.4 → 22 balls → 3.667 overs). Rounded to 2 decimal places.
- **Balls_Bowled** — Integer. Overs_Bowled converted from over.ball notation to total balls (e.g., 3.4 → 22). Overs are always summed as balls, so 3.4 + 3.4 correctly gives 7.2 overs rather than 6.8.
- **Batting_Order** — Text. Batting_Position grouped into top (positions 1–3), middle (4–7), or lower (8–11) order.

7. Analytical Modules

7.1 Batting Analytics

Individual Runs Scored

Aggregates total runs and balls played per player across all matches. Filtered to players who faced at least one ball. Sorted highest to lowest and displayed as a horizontal bar chart with a warm gold-to-red colour palette.

Total Runs Contribution

Shows each player's share of the team's total runs as a pie chart. The top 5 scorers are shown individually; remaining players are grouped as 'Others'. Only players with runs greater than 0 are included.

Player's Total Runs in Each Match

A line chart showing the run-scoring trend for a selected player across matches. Available only when the dataset contains more than one match. The player is selected via a dropdown menu. The y-axis starts at zero with evenly spaced whole-number ticks, so small changes are not visually exaggerated.

Runs Contribution by Role

Groups total runs scored by player role (batsman, bowler, all-rounder) and displays as a pie chart. This reveals which role group is carrying the most batting responsibility.

Strike Rate Comparison — Batsmen vs All-Rounders

Compares the average strike rates of batsmen and all-rounders who have faced 10 or more balls in total. Colour-coded by role. Helps identify which role group is scoring more aggressively.

Player Consistency in Batting

Calculates a consistency score for each player who appears in more than one match. The formula is mean_runs / (std_runs + 1). A higher score indicates both higher average runs and lower match-to-match variance. Only available for multi-match datasets with repeated players.

Total Runs by Batting Order

Groups players into top order (positions 1–3), middle order (positions 4–7), and lower order (positions 8–11), then compares the total runs scored by each group.

Average Runs by Batting Order

Uses the same three groups but shows the average runs scored per batting innings, counting only innings where the player faced at least one ball. This separates how productive each position is from how often it gets to bat.

Phase-wise Wickets Lost

Categorises each dismissal into one of three phases — Powerplay (overs 1–6), Middle Overs (overs 7–15), or Death Overs (overs 16–20) — based on the Out_Over value. Displays total wickets lost in each phase as a bar chart.

7.2 Bowling Analytics

Individual Wickets Taken

Aggregates total wickets taken and overs bowled per player across all matches. Filtered to players who bowled at least one over. Sorted highest to lowest and displayed as a horizontal bar chart with a cool blue-teal palette. Overs are added as balls so partial overs are totalled correctly.

Total Wickets Contribution

Shows each player's share of the team's total wickets as a pie chart. Top 5 wicket-takers are shown individually; the rest are grouped as 'Others'.

Player's Total Wickets in Each Match

A line chart showing the wicket-taking trend for a selected player across matches. Available only when the dataset contains more than one match. The y-axis starts at zero with evenly spaced whole-number ticks.

Wickets Contribution by Role

Groups total wickets taken by player role and displays as a pie chart. Reveals whether the team's bowling is being driven by specialist bowlers or all-rounders.

Average Wickets by Role

Shows the average wickets taken per bowling innings for each role, counting only innings where the player actually bowled. It complements the role-wise totals by accounting for how many bowling innings each role group had.

Economy Rate Comparison — Bowlers vs All-Rounders

Compares the average economy rates of bowlers and all-rounders who have bowled 2 or more overs in total. Sorted from lowest (best) to highest economy rate. Helps identify the most economical bowling options.

Player Consistency in Bowling

Uses the same consistency score formula as batting — mean_wickets / (std_wickets + 1) — to identify players who deliver wickets consistently across matches rather than in isolated performances. Only available for multi-match datasets with repeated players. The calculation is based on bowling data only.

7.3 Player Analytics

A dedicated tab for single-player analysis. The user selects a player from a dropdown and sees an individual batting and bowling profile across all matches. The whole tab is a fragment, so changing the player or match re-runs only this tab.

Player Information

Insight lines showing the player's role (most common role), matches played, usual batting order (average batting position: top up to 3, middle up to 7, otherwise lower), total runs scored, and total wickets taken. Runs and wickets are shown only if the player batted or bowled.

Individual Batting Stats

Four KPI cards (Total Runs, Balls Played, Strike Rate, Batting Consistency), followed by runs in each match (table and bar chart), strike rate in each match (line chart), runs contribution in a selected match (pie chart against the rest of the batting line-up), and run contribution against the team's total across all matches (pie chart). If the player never batted, a message replaces this section.

Individual Bowling Stats

Four KPI cards (Total Wickets, Overs Bowled, Economy Rate, Bowling Consistency), followed by wickets in each match (bar chart), economy rate in each match (line chart), wickets contribution in a selected match (pie chart against the rest of the bowling attack), and wicket contribution against the team's total (pie chart). Economy is calculated from total balls bowled. If the player never bowled, a message replaces this section.

7.4 Summary Tab

Total Runs and Wickets per Match

Side-by-side bar charts showing the team's total runs scored and wickets taken in each match. Gives an at-a-glance view of team-level performance trends across the series.

Top Performers KPI Cards

Four KPI cards displaying the top performer in each key metric:

- Top Run Scorer — with total runs and balls faced.
- Top Wicket Taker — with total wickets.
- Best Strike Rate — among batsmen and all-rounders.
- Best Economy Rate — among bowlers and all-rounders.

Team Performance Summary

Eight auto-generated narrative insight lines covering: top scorer, top wicket-taker, best strike rate, best economy rate, highest run-contributing role, highest wicket-contributing role, strongest batting order segment, and the phase where the team loses the most wickets.

8. Tech Stack

The following tools and libraries are used in this project:

Tool	Version	Purpose
Python	3.14	Core language
NumPy	Any recent	Numerical operations
Pandas	Any recent	Data loading, cleaning, aggregation
Matplotlib	Any recent	Base charting library
Seaborn	Any recent	Statistical visualisation and palette management
Streamlit	1.57.0	Web application framework and UI

External resources loaded at runtime:

- Google Fonts — Rajdhani and DM Sans, loaded via CDN for UI typography.

9. Version History

Version 1.0 — 27th February 2026

- Initial release.
- Jupyter Notebook-based analysis using 3 RCB IPL 2024 match datasets.
- Basic data validation, cleaning, and feature engineering.
- Static charts for batting and bowling analytics.
- Plain-text 8-point summary output.

Version 2.0 — 16th March 2026

- Complete rewrite as a Streamlit web application.
- Added .xlsx upload support alongside .csv.
- Multi-file upload support with automatic concatenation.
- Enhanced data validation with additional edge-case checks.
- Interactive player-level trend charts with dropdown selectors.
- Dark-themed dashboard UI with tabbed navigation.
- Improved visualisation clarity and chart styling.

Version 2.01 — 18th April 2026

- Improved overall tool user interface with refined CSS theming.
- Added KPI cards for key performance indicators in the Summary tab.
- Added expandable sections for Dataset Columns Overview and About page.
- Improved data validation and cleaning logic.
- Improved line chart logic and runs/wickets contribution breakdowns.
- Fixed various bugs and edge cases.

Version 2.1 — 16th September 2026

- Added a new validation rule: the total wickets taken in a single match cannot exceed 10.
- Added the Player Analytics tab with individual batting and bowling profiles, KPI cards, per-match charts, and contribution pie charts.
- Split batting order runs into Total Runs and Average Runs by Batting Order sections, and added Average Wickets by Role.
- Fixed validation so a blank Out_Over is accepted for not-out batters, and corrected an operator-precedence bug in the bowling-column check.
- Overs are now summed as balls so partial overs total correctly; bowling consistency now uses bowling data.
- Line and bar charts now use evenly spaced whole-number y-ticks with a zero baseline instead of one tick per data value.
- Performance improvements: fragments for interactive sections, cached shared statistics, and reusable chart helpers.
- Renamed the tool to Cricket Team Performance Analyzer (currently only supports the T-20 format) and replaced the three RCB example datasets with one realistic 10-match dataset.
- Fixed row numbering in filtered tables, and empty match phases now show 0 instead of NaN.

10. Known Limitations

The following limitations are present in the current version of the tool:

- T20 format only — Out_Over values cannot exceed 20, and phase definitions are fixed to T20 boundaries.
- A single upload session is limited to 22 unique players (2 complete teams). Longer series requiring squad rotations must be split and analysed separately.
- The consistency score metric requires a player to appear in at least 2 matches. Single-match datasets or those with a completely different lineup per match will not produce consistency results.
- The tool does not account for extras (wides, no-balls, byes) as separate fields. Extras attributed to a bowler should be manually included in Runs_Given.
- Run-outs are not tracked as a separate dismissal type. They count toward Was_Out based on Out_Over but are not attributed to any bowler in Wickets_Taken.
- Values like 2.6 or 4.7 in overs notation (invalid ball counts) are not explicitly caught in validation and may produce incorrect economy rates.
- The tool does not support fielding or wicketkeeping statistics — catches, stumpings, and run-out assists are not tracked.

11. Future Scope

The following enhancements are planned for future versions of the tool:

- Format support — ODI (50-over) and Test match formats, with adjusted phase definitions and validation rules.
- Opposition data — Adding the ability to input opposition team data alongside your own team for head-to-head comparison.
- Match result tracking — Adding a Result field (Win/Loss) to enable win-correlation analysis, such as identifying which players perform better in winning matches.